

Lab 6 – Building a Responsive Movie Genre Browsing Screen

1. Objective

In this lab, students will design and implement a **responsive movie browsing screen** that adapts to different screen sizes (small phones, large phones, tablets, and web).

The lab reinforces the key ideas from **Module 6 – Responsive UI & Adaptive Layouts**, including:

- Using **MediaQuery** to read screen dimensions
- Using **LayoutBuilder** to adapt to available space
- Using **Wrap**, **Expanded**, and **ListView/GridView** to build responsive layouts
- Building a real-world **Genre Filtering Screen** similar to the one shown in Figures 6.1–6.9 of *Mastering Flutter 2025*.

At the end, students will have a **fully functional responsive screen** that runs correctly in:

- DartPad (Flutter)
- Android Studio
- VS Code

2. Requirements

This lab is structured into three sub-tasks inside a single project:

- **Lab 6.1 – Responsive Hero & Heading Section**
- **Lab 6.2 – Search, Genre Chips & Sort Bar**
- **Lab 6.3 – Responsive Movie List & Tablet Layout**

You may complete these progressively in the same `main.dart` file.

2.1 Functional Requirements

Your responsive movie browsing screen must:

1. Display a title heading, e.g. **“Find a Movie”**.
2. Include a **search bar** that allows users to type a movie title or keyword.
3. Display a **set of genre chips** (Action, Drama, Comedy, etc.) that can be toggled on/off.
4. Include a **sort dropdown** with at least the following options:

- A–Z
 - Z–A
 - Year
 - Rating
5. Show a **list of movie cards** for matching movies, including:
- Poster image (use a simple network placeholder image)
 - Title
 - Year
6. Filter the visible list of movies based on:
- Search text (case-insensitive match in title)
 - Selected genres (if any genre is selected, only show movies that have at least one of those genres)
7. Adapt the layout responsively:
- On **small screens**: movie cards stacked in a **single column list**
 - On **wider screens (e.g. ≥ 800 px)**: movie cards arranged in **two columns** (e.g. using `GridView.count`)

2.2 Technical Requirements

- Use Flutter and Dart only (no third-party packages).
- The entire demo must run from a **single file** (e.g. `main.dart`), so it can be pasted into **DartPad** → **New Flutter**.
- Use **MediaQuery** and/or **LayoutBuilder** to implement breakpoints, not hard-coded “device names”.
- Use **SafeArea** to avoid notches / camera cutout issues.
- Use **Wrap** for genre chips so they automatically wrap to the next line.
- Code must compile and run without errors in:
 - DartPad (Flutter mode)
 - Android Studio

- VS Code

2.3 Optional Enhancements (Bonus)

You may optionally:

- Add a small “badge” with the number of selected genres.
- Display movie rating as stars or numeric text.
- Add a “Clear filters” button.
- Provide different poster sizes for tablet vs phone using `LayoutBuilder`.

These are not required for passing the lab but can improve your score.

3. Guided Steps

Step 1 – Project Setup

1. Create a new Flutter project (e.g. `flutter create lab6_responsive_ui`) or open **DartPad** → **New Flutter**.
2. Replace the default `main.dart` content with the **Full Demo Code** at the end of this lab (or start from an empty `MaterialApp` and follow the steps).

Step 2 – Define the Movie Model & Sample Data

1. Create a simple `Movie` class with fields:
 - `title (String)`
 - `year (int)`
 - `genres (List<String>)`
 - `posterUrl (String)`
 - `rating (double)`
2. Define a constant list of sample movies (3–6 items) in a variable like `allMovies`.

This allows you to build UI first without backend/API.

Step 3 – Build the Base Scaffold

1. Create a `ResponsiveMovieApp` widget that returns a `MaterialApp`.

2. Set home to a GenreScreen (stateful widget).
3. Inside GenreScreen, wrap content in SafeArea and Padding.

The basic layout should look like:

- Title: “Find a Movie”
- Then vertical column for search bar, genres, sort, and movie list.

Step 4 – Implement a Responsive Search Bar

1. Add a TextField wrapped in a rounded container to simulate a search bar.
2. Add a state variable String searchQuery = "";
3. In the onChanged callback of the TextField, update searchQuery using setState.
4. Use searchQuery later in your filtering logic to filter movies by title.

Step 5 – Implement Genre Chips Using Wrap

1. Declare a List<String> genres = [...] with some default genres.
2. Add a Set<String> selectedGenres = {}; or List<String> to track selected items.
3. Use a Wrap widget to display all genres as tappable chips.
4. When a chip is tapped:
 - If it was selected, remove it from selectedGenres.
 - Otherwise, add it.

The Wrap widget will automatically wrap chips to the next line, making it responsive.

Step 6 – Implement Sort Dropdown

1. Create an enum or string values to represent the sort options (A–Z, Z–A, Year, Rating).
2. Add a DropdownButton<String> bound to a state variable (e.g. selectedSort = "A-Z").
3. In the onChanged callback, update the sort variable and call setState.
4. Later, apply sorting to the filtered list before displaying it.

Step 7 – Filter and Sort the Movie List

1. Build a computed list (e.g. List<Movie> visibleMovies) inside build() based on:

- Search query
 - Selected genres
2. Then sort `visibleMovies` according to `selectedSort`.
 3. You can use `.where()` and `.toLowerCase()` for filtering, and `.sort()` for ordering.

Step 8 – Build a Responsive Movie List (ListView + LayoutBuilder)

1. Wrap the movie list area in an Expanded widget inside the main Column.
2. Use a LayoutBuilder to decide layout based on `constraints.maxWidth`:
 - If `maxWidth < 800`: use a `ListView.builder` with a single column.
 - Else: use a `GridView.count` with `crossAxisCount: 2`.
3. Each movie item should show:
 - Poster image (`Image.network`)
 - Title
 - Year

Use `LayoutBuilder` again inside a movie card to adjust poster size based on item width.

Step 9 – Test on Different Devices / Viewports

1. In DartPad:
 - Resize the preview pane
 - Switch orientation
2. In Android Studio / VS Code:
 - Run on a small phone emulator
 - Run on a larger phone
 - Run on a tablet emulator / Chrome Web

Confirm that:

- Chips wrap properly
- Movie list layout changes at the breakpoint

- Nothing overflows or gets clipped

4. Expected Result

When you finish the lab, your app should:

- Show a title “Find a Movie”.
- Display a search bar that filters the list by movie title (case-insensitive).
- Display a genre chip section at the top; selecting chips filters the list by genre.
- Provide a sort dropdown that affects the order of displayed movies.
- Adapt the layout based on screen width:
 - **Phones:** single-column vertical list of movie cards.
 - **Tablets / Wide web windows** (≥ 800 px): two-column movie layout.
- Match the look & feel of the Genre Screen from Figures 6.1–6.9 in *Mastering Flutter 2025* (not pixel-perfect, but functionally similar).

5. Submission

Deliverable:

- A zipped Flutter project *or* a **DartPad Flutter share link** containing the completed responsive screen.

Evaluation Criteria:

- **Responsive Layout Logic (30%)**
Correct use of MediaQuery / LayoutBuilder and breakpoints for phone vs tablet.
- **UI Composition & Design (30%)**
Clear layout with readable text, chip design, and movie cards.
- **Filtering & Sorting Functionality (25%)**
Search, genre selection, and sort options work as expected.
- **Code Quality & Clarity (15%)**
Clean, readable code with meaningful names and comments.